PROJECT EVALUATION DOSSIER

ReliaDL

Verification as a Control Signal: Integrity-Aware Scheduling for Resilient Parallel File Transfer

Everything a presenter needs for a jury evaluation of this project: the problem it solves, how it works, the mathematics and the complexity of every algorithm in it, what the measurements do and do not show, an unflinching account of its weaknesses, and the cross-questions a jury is likely to ask with answers that will survive follow-up.

Field	Detail
Artefact	ReliaDL - parallel chunked HTTP range-request download engine
Scale	15,920 lines of Python across 32 test modules and 1,009 tests
Core claim	Cryptographic verification outcomes are used as a scheduling control signal, not only as a terminal pass/fail gate
Evidence base	690 seeded benchmark runs; 7 scenarios x 5 policies x 30 seeds; raw data committed
Companion paper	10 pages, IEEE two-column, 8 figures, 7 tables, 22 equations
Status	Working system; evaluation complete but bounded (see Section 9)

1 Executive summary

A parallel downloader splits a file into byte ranges and fetches them concurrently from one or more servers. Every such tool - aria2c, axel, curl with ranges, and the research systems built on the same idea - makes its scheduling decisions from throughput and latency alone. How large should the next chunk be? Which mirror should serve it? Both questions are answered from speed. Integrity, when it is checked at all, is a gate applied after the bytes have landed: hash the file, compare, pass or fail.

That separation is the problem this project attacks. A server that returns the wrong bytes is not a slow server; it is a different kind of fault, and treating it as a speed problem produces a specific and measurable failure. In our benchmarks a configuration with no per-chunk digests completed 0 out of 5 runs against a corrupting mirror. Not slowly - not at all.

The contribution is to close the loop. A verification outcome is fed back into two scheduling decisions it has never previously influenced: the size of the next chunk, and the source the next chunk is requested from. Three pieces follow from that, each one independently measurable: a chunk-sizing cost model corrected for worker concurrency, an integrity-coupled source-selection policy, and sub-block localization that bounds the cost of a corrupt chunk.

The honest summary of the results is that the integrity result is large and the performance results are mixed. Sub-block verification turns a transfer that fails into a transfer that succeeds, every time. Integrity-coupled routing cuts wasted bytes about fourfold. The concurrency floor on chunk sizing is worth between 16% and 147% depending on the scenario. But adaptive chunk sizing is indistinguishable from a well-chosen fixed chunk size on every static path we test, helping only where the path degrades mid-transfer, mirror routing costs 42% on a scenario where the origin is already the fastest source available, and aria2c moves unimpaired bytes about 1.6 times faster than we do. Those negative results are reported here with the same prominence as the positive ones, because a jury that discovers them itself will trust nothing else in the document.

> **The one-sentence pitch**
>
> Parallel downloaders decide what to fetch and where to fetch it from using speed alone, and verify integrity only at the end; we make verification a first-class input to those decisions, and show that doing so is the difference between a transfer that completes and one that cannot.

2 The problem

2.1 What actually goes wrong

Large file transfer over HTTP is routine and is assumed to be solved. It is not, in three specific ways that compound when a transfer is parallel and multi-source.

- **Silent corruption is not caught by the transport.** TLS protects a connection against an on-path attacker and TCP checksums catch a fraction of random bit errors. Neither says anything about a server that reads the wrong bytes off its own disk, a CDN edge holding a stale or partially-written object, a misconfigured transcoding proxy, or a compromised mirror. In every one of those cases the bytes arrive perfectly intact over a perfectly valid TLS session, and they are wrong.
- **Detection without attribution is almost useless.** A whole-file checksum tells you the 64 MB you just downloaded is bad. It does not tell you which chunk, which mirror, or which byte. The only available

recovery is to discard everything and start again - from a source set you have no new information about, so the second attempt is as likely to fail as the first.

- **Parallelism multiplies the exposure.** A single-stream download from one origin has one source to trust. A four-worker download across three mirrors has three, and the fastest of them will be given the most work by any throughput-driven scheduler - including, as we show in Section 6.2, a scheduler that is already being told about verification failures.

2.2 Who has this problem

This is not a hypothetical. The populations that care are the ones where a corrupt artefact is worse than a slow one: software and container distribution, where a bad binary is a supply-chain event; scientific data transfer, where multi-terabyte datasets move between institutions over paths with no end-to-end integrity story; machine-learning weight distribution, where a corrupted checkpoint fails silently and expensively; and anyone fetching from community mirrors, where the operator of the fastest mirror is not necessarily someone you have a trust relationship with.

2.3 Why the existing tools do not solve it

Figure 1 places the relevant systems on two axes: how finely integrity is checked, and how strongly a verification outcome changes a later scheduling decision. The honest reading of this figure is the important one - it is not that prior work ignores integrity, it is that the feedback is either absent or coarse.



Figure 1. Where this work sits. BitTorrent is deliberately placed high: it has verified pieces and banned misbehaving peers for two decades, and any claim that integrity-driven source selection is new would be false. The unoccupied region is sub-chunk granularity in the HTTP range-request setting, with chunk size itself treated as a control action.

- **curl, wget, axel** - no integrity checking of any kind. A verification outcome does not exist to feed back.
- **aria2c** - supports a whole-file checksum and, with a Metalink file, per-piece hashes. Routing and piece sizing remain throughput-driven; a hash failure triggers a retry, not a change of policy.
- **FIVER** (arXiv:1811.01161) - the closest prior work. It verifies chunk digests during the transfer rather than after it, which is exactly the right move, but the outcome drives discard-and-refetch, not scheduling.

- **BitTorrent** - per-piece SHA-1 verification with peers banned on failure. This is genuine integrity-driven source selection and we say so plainly. It differs from this work in setting (swarm, not HTTP range requests), in granularity (a whole piece, typically 256 KB to 4 MB, is discarded), and in that piece size is fixed by the torrent at creation time and is never a control variable.

> **The gap, stated precisely**
>
> No prior system treats the chunk size of an HTTP range-request transfer as a control action whose optimum depends on the verification failure rate and the localization granularity. That is the specific thing that is new here, and it is a smaller claim than 'integrity-aware downloading', which would not be true.

3 The idea, and how the system is built

3.1 The control loop

Figure 2 is the architecture reduced to the one edge that matters. Everything drawn thin is conventional and present in any parallel downloader. The heavy edge - verification outcome back into the scheduler - is the project.

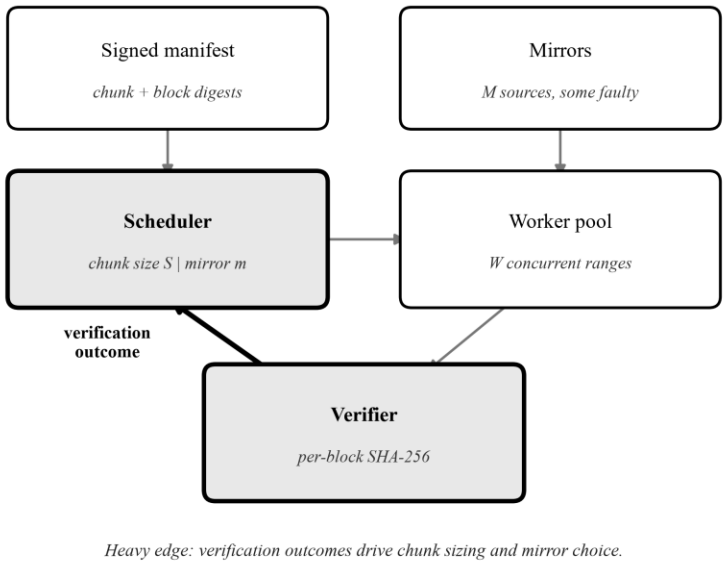
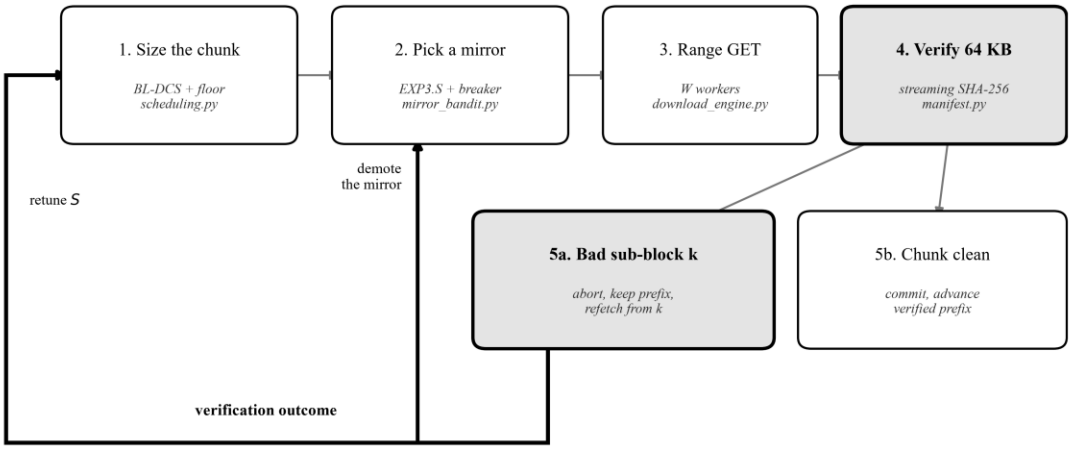


Figure 2. The control loop. A signed manifest supplies per-chunk and per-sub-block digests; the scheduler chooses a chunk size and a mirror; W workers issue range requests; the verifier checks each 64 KB sub-block as it arrives. The heavy edge returns that verdict to the scheduler.

3.2 The life of one chunk

Figure 3 unrolls the loop into the sequence a single chunk actually follows, with the owning module named at each step so a question of the form 'where is that in the code' has an immediate answer.



The heavy path is the contribution: a verification outcome changes the next chunk's size and its source.

Figure 3. One chunk from planning to commit. The feedback bus along the bottom carries the verification outcome to the two decisions it influences. A failure both demotes the mirror that served it and raises the measured failure rate the chunk-size controller consumes.

3.3 Module map

The system is 15,920 lines of Python. The table below is the part a jury needs; it also states which modules are on the live download path and which are library code, because an earlier version of this project's documentation did not make that distinction and the result was a set of claims about algorithms that were not reachable from any execution path.

Module	Responsibility	On the live path
download_engine.py	Worker pool, range requests, retry, checkpointing	Yes
scheduling.py	The seam: chunk sizing, mirror choice, failure accounting	Yes
manifest.py	Merkle tree, Ed25519-signed manifests, sub-block validation	Yes
algorithms/adaptive_chunker.py	BL-DCS square-root chunk-size controller	Via scheduling.py
algorithms/mirror_bandit.py	EXP3.S router and the circuit breaker	Via scheduling.py
algorithms/metrics_collector.py	EWMA RTT and throughput, sliding failure window	Yes
algorithms/work_stealer.py	Straggler detection, range bisection	Library
algorithms/task_deque.py	Lock-based work-stealing deque	Library
state_manager.py	Crash-safe resume state	Yes
sparse_writer.py	Positional writes into a sparse output file	Yes
hash_verifier.py	Streaming SHA-2, constant-time comparison	Yes

4 The three technical contributions

Each is stated with its mathematics, its provenance, and the measurement that supports or fails to support it. Provenance matters here: two of the three rest on textbook prior art, and the contribution is what is added to it, not the base algorithm.

4.1 C1 - Chunk sizing under a concurrency constraint

The classical model

A chunk that is too small pays per-request overhead on every range GET. A chunk that is too large wastes more bytes when it has to be re-fetched. Writing tau for per-request overhead, mu for throughput, p_seg for the per-segment drop probability and MSS for the maximum segment size, the per-byte cost is

$$J(S) = tau \,.\, mu \,/\, S \;+\; p_seg \,.\, S \,/\, MSS \qquad (1)$$

Setting dJ/dS = 0 gives the familiar square-root optimum,

$$S^* = sqrt(\, tau \,.\, mu \,.\, MSS \,/\, p_seg \,) \qquad (2)$$

This is the same first-order argument that produces the economic order quantity, and we claim no novelty in it. The MSS factor inside the radical is what makes the radicand a byte-squared quantity so the root yields bytes.

What is ours, part one: the per-segment rescaling

The metrics collector observes failures per chunk; the cost model is expressed per segment. At an 8 MB chunk those two rates differ by roughly four orders of magnitude, so feeding one into the other is dimensionally incoherent even though the formula looks correct. The conversion uses the transfer size that produced the observation:

$$p_seg = 1 - (1 - p_chunk)\ \hat{}\ (MSS \,/\, S_observed) \qquad (3)$$

What is ours, part two: the concurrency constraint

Equation (1) describes a single sequential stream. Nothing in it mentions parallelism, and that omission dominates in practice. Aggregate throughput for a parallel downloader is roughly the number of chunks in flight times the per-connection rate. A chunk large enough that fewer than W of them remain leaves some of the W workers idle, and the parallelism lost costs far more than the per-request overhead saved.

The unconstrained controller walks straight into this. On a clean path it observes no loss, grows the chunk towards its 64 MB ceiling, and starves the pool. The fix caps the issued chunk so that at least W chunks always remain:

$$S = min(\, S_BLDCS \,,\ max(\, S_min \,,\, F_rem \,/\, W \,)\,) \qquad (4)$$

The cap only ever shrinks a chunk, so the loss-driven lower bound the controller computes is untouched. It constrains the solution without replacing the model. Figure 4 shows why this matters: the unconstrained optimum for the controller's own default parameters on a clean path lies deep inside the region where the worker pool would be starved.

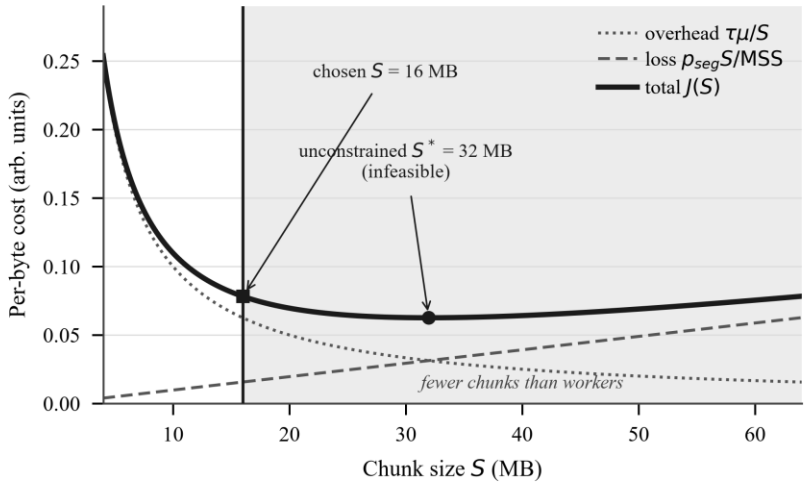


Figure 4. The cost model of Equation (1) with the feasibility limit of Equation (4) overlaid. The shaded region is where fewer chunks remain than there are workers. The unconstrained minimum sits inside it; the chosen size is the constrained one.

This is the one genuinely new piece of modelling in the project. The square-root tradeoff is textbook. Stating that it is the wrong objective for a parallel downloader unless it is constrained by the worker count, and measuring the size of the error, is not something the chunk-sizing literature appears to cover.

4.2 C2 - Integrity-coupled source selection

Why an adversarial bandit

When a file is available from several CDN edges or replicas, an edge's throughput depends on other tenants' load, on routing changes and on cache state - none of which are stationary and none observable. Only the reward of the mirror actually used is observed. That is a bandit problem, and the non-stationarity motivates an algorithm that assumes nothing about the reward distribution. We use EXP3.S (Auer, Cesa-Bianchi, Freund and Schapire, 2002), which is prior art; the implementation and the decay measurement are ours, the algorithm is not.

$$p(m,t) = (1 - eta) \cdot w(m,t) / SUM_j\ w(j,t) \ + \ eta / M \qquad (5)$$

$$r_hat = r / p(m,t) \qquad w(m,t+1) = w(m,t) \cdot exp(eta \cdot r_hat / M) \qquad (6)$$

The uniform eta/M floor keeps every mirror under occasional test and bounds every selection probability away from zero, which is what makes the importance-weighted estimate finite. Weights are held as logarithms and rescaled so the largest is exactly one - a requirement rather than a refinement, since the per-round exponent accumulates and a long download makes tens of thousands of range requests.

EXP3.S adds a decay that mixes each weight back toward the mean, capping how much evidence any arm can hoard:

$$w_m \ <- \ (1 - alpha) \cdot w_m \ + \ (alpha / M) \cdot SUM_j\ w_j \qquad (7)$$

Over 24 seeded trials of 5,000 rounds across stationary, abrupt-failover and drifting reward schedules, classic EXP3 (alpha = 0) achieved the lowest regret under stationarity and the worst by far elsewhere (1709 against 217). Any small positive decay cut worst-case regret more than fourfold. The optimum sat at alpha = 0.0002, which is 1/T for the horizon measured, consistent with the EXP3.S guidance, and was confirmed at T = 1000, 2000 and 10000.

The result that reorganised the project

The natural argument is that no extra mechanism is needed: EXP3.S already scores a failed request as zero reward, a chunk failing verification is a failed request, so the router should demote a corrupting mirror for the same reason it demotes a broken one. We believed this, wrote it down, and then ran it.

It is false. Against three mirrors where the corrupting one is twice as fast and corrupts 40% of what it serves, averaged over 40 seeded trials of 400 requests, integrity-aware reward alone sends 45.5% of requests to the corrupting mirror. Uniform allocation over three mirrors would be 33%. The reward does not merely fail to demote the bad mirror - it actively prefers it.

The reason is visible once the expected reward is written out. A mirror delivering throughput r_f but corrupting a fraction q of what it serves beats an honest mirror delivering r_h whenever

$$q < 1 - r_h / r_f \qquad (8)$$

A mirror twice as fast can therefore corrupt up to half of everything it serves and remain the best arm by expected reward. What actually displaces it is the consecutive-failure circuit breaker, which reacts to the rate of failures rather than to their average: adding it drops the corrupting mirror's share from 45.5% to 5.4%. Figure 5 shows both arms.

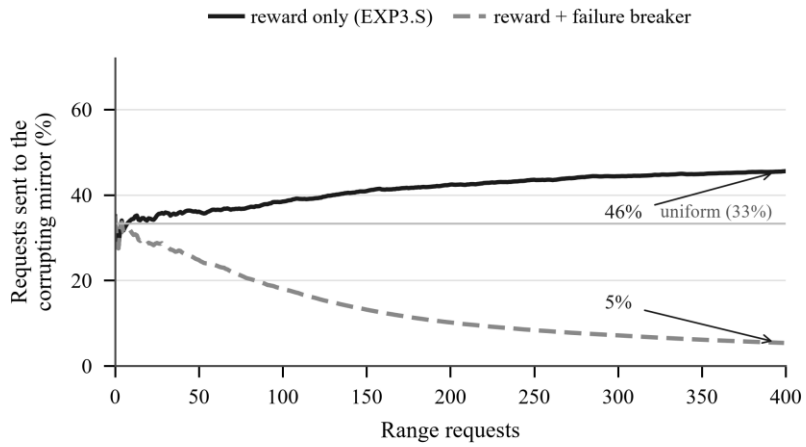


Figure 5. Share of requests sent to a corrupting mirror over 400 range requests, reward-only against reward-plus-breaker, run against the shipped EXP3Bandit and MirrorDispatcher. The horizontal reference is uniform allocation over three mirrors, 33%. Reward alone converges above it.

The general lesson, and the sentence to lead with if a juror asks what you learned: an average-reward learner is the wrong instrument for an integrity fault, because integrity failures are not noise in the reward - they are a property the reward deliberately averages away.

4.3 C3 - Sub-block localization and its honest bound

When a manifest supplies per-sub-block digests, default 64 KB, each chunk is streamed through a validator that checks each sub-block as it completes. A corrupt chunk is abandoned at the first bad block rather than after the whole range has arrived, and the verified prefix is kept; the retry re-requests only from the bad sub-block onward. For a single flipped byte in sub-block 40 of a 64-sub-block, 4 MB chunk, whole-chunk verification re-fetches 4096 KB and localization re-fetches 1536 KB.

The saving is position-dependent, and this is the part that must be stated before a juror finds it. Because the retry resumes from the bad sub-block rather than re-fetching only that block, bytes re-fetched for a corruption in sub-block k of a chunk of B bytes with sub-block size s are

$$W(k) = B - k \,.\, s \qquad (9)$$

so corruption in the last sub-block saves nearly the whole chunk, corruption in sub-block 0 saves nothing at all, and uniformly distributed corruption saves about half the chunk on average:

$$E[W] = B / 2 \; + \; s / 2 \,, \quad \text{for } k \text{ uniform on } \{0, \ldots, B/s - 1\} \qquad (10)$$

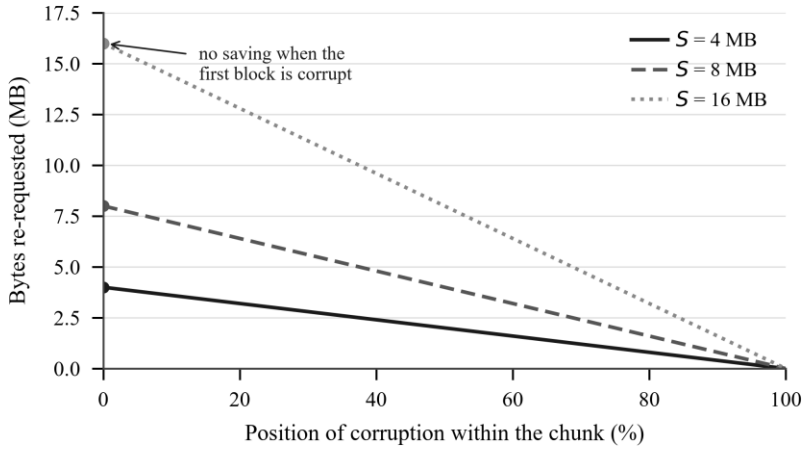


Figure 6. Bytes re-requested as a function of where the corruption falls within the chunk, Equation (9), for three chunk sizes. The marked left-hand end is the case that matters: localization saves nothing at all when the first sub-block is the corrupt one.

An earlier revision of this project's documentation claimed a 99.95% saving, derived by assuming the retry re-fetches only the bad sub-block. That figure was wrong and the gap is structural, not an implementation detail. Reaching a position-independent cost of one sub-block per corruption would require not aborting: verify the whole chunk, collect every bad sub-block, then re-fetch exactly those ranges. That trades the bandwidth saved by aborting early against a smaller retry, and it also slows detection of a corrupting mirror because the stream runs to completion before the verdict. Which side wins depends on corruption rate, chunk size and sub-block size; it is unmeasured here and is the most interesting open question in the project.

Detection latency is the other half of the tradeoff, and it runs the opposite way. A mirror that corrupts a fraction q of the responses it serves fails a verification check on each response independently with probability q, so the number of chunks it serves before its first detected failure is geometric with mean 1/q, and the bytes it is allowed to serve before being caught are

$$E[\text{ bytes before detection }] = S / q \qquad (11)$$

Large chunks are good for goodput and bad for detection. At the byzantine scenario's q = 0.4, a 16 MB chunk lets a corrupting mirror serve 40 MB before it is first caught, against 2.5 MB for a 1 MB chunk. This is the tension at the centre of the project: chunk size is simultaneously a throughput parameter and a security parameter, and no prior downloader treats it as both.

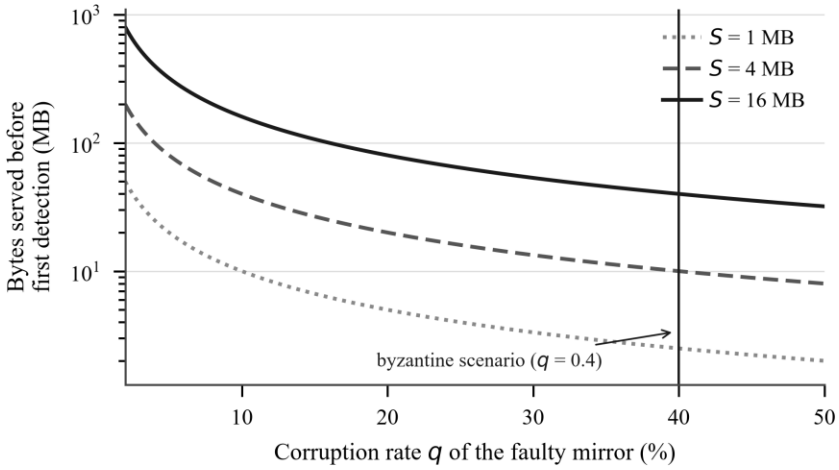


Figure 7. Bytes a corrupting mirror serves before its first detected failure, Equation (11), against its corruption rate, for three chunk sizes. Logarithmic vertical axis.

4.4 Integrity construction

The manifest is a binary Merkle tree over chunk hashes with domain separation - SHA-256(0x00 || leaf) for leaves and SHA-256(0x01 || left || right) for internal nodes - canonicalized per RFC 8785 and signed with Ed25519. The published root additionally commits to the leaf count:

$$root = SHA\text{-}256(\ 0x02 \ ||\ uint64_be(leaf_count) \ ||\ tree_root\ ) \qquad (12)$$

This closes a real second-preimage weakness that was present in the original implementation. Odd nodes are paired with themselves, so the leaf lists [a, b, c] and [a, b, c, c] built identical internal nodes and therefore the same root - and the manifest signs that root. The collision was verified to exist before the fix and to be closed after it. Binding the count was chosen over the Certificate Transparency promotion rule because it leaves the tree shape, and therefore every existing audit path, intact.

5 Algorithmic complexity

Notation: N is the file size in bytes, S the chunk size, B the number of chunks (N/S), s the sub-block size, W the number of concurrent workers, M the number of mirrors, and T the number of scheduling rounds. Costs are stated per the unit named in the third column, which is where most complexity questions actually go wrong.

Operation	Time	Per	Space	Notes
Merkle tree construction	O(B)	transfer	O(B)	B hash evaluations, one per leaf plus one per internal node
Inclusion proof generation	O(log B)	chunk	O(log B)	32 bytes per level; 20 levels for a million chunks
Inclusion proof verification	O(log B)	chunk	O(1)	Constant-time root comparison
Leaf-count root binding	O(1)	transfer	O(1)	One extra hash over 9 bytes
BL-DCS chunk size	O(1)	decision	O(1)	Closed form; stateless by construction
Concurrency floor	O(1)	decision	O(1)	One division and one comparison
EXP3.S arm selection	O(M)	round	O(M)	log-sum-exp over the active arm set
EXP3.S weight update	O(M)	round	O(M)	The decay mixes toward the mean, so all weights are touched
Circuit breaker check	O(1)	request	O(M)	Per-mirror consecutive-failure counter and cooldown clock
Sub-block streaming verify	O(S)	chunk	O(s)	One SHA-256 pass over the bytes already in flight
Digest comparison	O(S/s)	chunk	O(1)	Constant-time compare per sub-block
Chunk planning	O(1) amortized	chunk	O(W)	Lazy, planned in batches of max(8, W)
Straggler detection	O(W)	sweep	O(W)	Scans active transfers only; idle workers excluded
Range bisection	O(1)	straggler	O(1)	Split at the midpoint of the unfetched remainder
Resume checkpoint	O(B)	checkpoint	O(B)	Full state rewrite, batched every 16 chunks or 5 s
Whole-file verification	O(N)	transfer	O(1)	Disk re-read; streaming, constant memory
Whole transfer	O(N + B log B)	transfer	O(B + W.buf)	Dominated by the O(N) byte path

5.1 The three complexity facts worth memorising

- **Memory is bounded by the write buffer, not by the chunk size.** Each worker coalesces writes into a 1 MB buffer and flushes; it never holds a whole chunk in memory. Peak client memory is therefore O(B + W x 1 MB) regardless of whether chunks are 4 MB or 64 MB - the O(B) term is the resume state, and it is the one that grows with file size.
- **Per-chunk checkpointing was the real bottleneck, and it was not algorithmic.** The original engine performed an fsync and a full state rewrite synchronously inside the worker coroutine after every chunk, serialising the entire transfer behind disk I/O on the event loop. Batching it to every 16 chunks and offloading it to a thread produced a measured 1.63x speedup with no change to any algorithm. The cost is bounded and paid only after a crash: at most 16 chunks are re-downloaded on resume, and no chunk is recorded complete before its bytes are durable.
- **Routing cost is linear in mirrors, and that is fine.** EXP3.S is O(M) per round because the decay touches every weight. For realistic M - a handful of CDN edges or distribution mirrors - this is irrelevant next to a network round trip. It would matter for a swarm with thousands of peers, which is not this setting, and we would say so rather than claim the design scales there.

6 Evaluation

6.1 Method

- **Workload.** A 64 MB payload generated from the run seed, fetched to a temporary file and verified against its SHA-256.
- **Origins.** One to three fault-injecting mirror servers on loopback, each with a profile fixing bandwidth, latency, jitter, request-failure rate, silent byte-corruption rate, truncation rate, and optionally a degraded profile adopted after a byte threshold.
- **Determinism.** Impairments are drawn from blake2b(seed, byte_range, attempt), so they depend on which bytes are requested and not on the order requests arrive. Two policies fetching the same file meet the same faults, which is what makes an ablation a comparison rather than a coincidence.
- **Repetitions.** Thirty seeds per cell, reported as median with interquartile range. Paired comparisons add a bootstrap 95% interval and an exact sign test, and are run interleaved so machine drift falls on both arms. Transfer times are right-skewed, so a mean and standard deviation would describe the distribution badly.
- **Waste** is measured at the server: bytes the mirrors wrote, minus the file size. It is measured there because a client that abandons a stream early cannot see the bytes it avoided.

Five policies were compared, each differing from the baseline in exactly one respect so that a difference is attributable to that one change: baseline (fixed 4 MB, origin only, no sub-block digests); adaptive-chunks; adaptive-mirrors; localization; and routing+localization. There is deliberately no 'everything at once' policy, because adaptive sizing and sub-block localization cannot be combined - a manifest's digests are computed over specific chunk boundaries and adaptive sizing exists to move those boundaries mid-transfer. When a manifest is supplied the engine adopts its partition and switches the planner off.

6.2 Result 1 - integrity decides whether the transfer completes at all

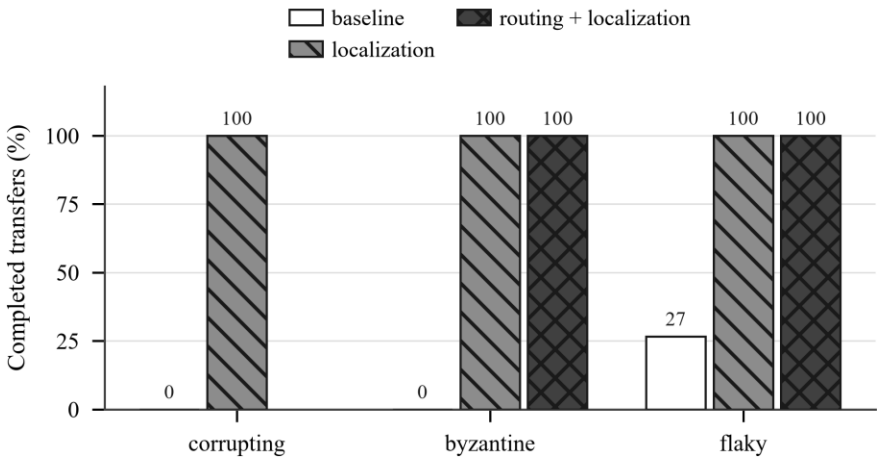


Figure 8. Completion rate out of five seeded runs. This is a correctness result, not a speed result.

Scenario	baseline	adaptive-chunks	adaptive-mirrors	localization	routing+localization
corrupting	**0 / 30**	**1 / 30**	-	**30 / 30**	-
byzantine	**0 / 30**	**0 / 30**	3 / 30	**30 / 30**	**30 / 30**
flaky	8 / 30	3 / 30	13 / 30	**30 / 30**	**30 / 30**

This is the clearest result in the set and it is the one to lead with. Without per-chunk digests from a manifest, silent corruption is caught only by the whole-file hash after everything has been transferred, and there is no way to attribute it to a chunk or recover from it. The transfer simply fails. With sub-block digests the corrupt chunk is repaired and the transfer completes every time.

6.3 Result 2 - routing away from a corrupting mirror cuts waste about fourfold

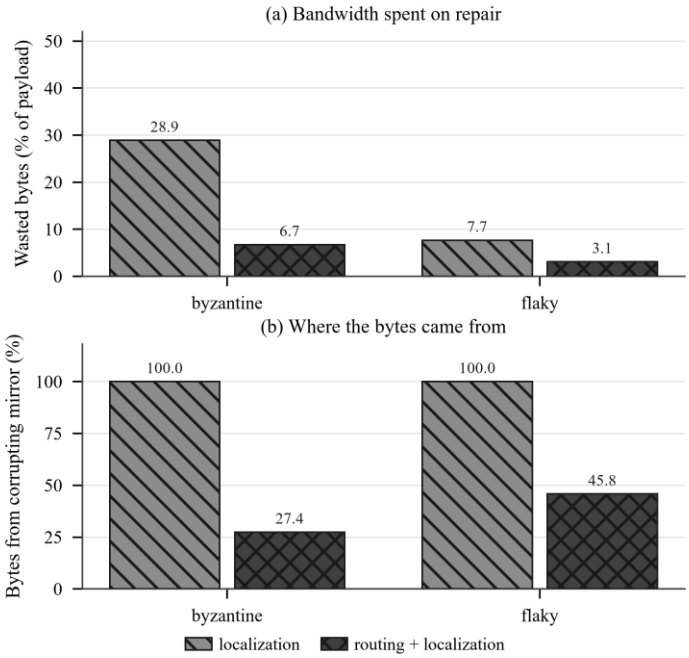


Figure 9. Wasted bytes (upper panel) and source attribution (lower panel) on the byzantine and flaky scenarios. On byzantine, three mirrors, the fastest corrupting 40% of what it serves.

Policy	Goodput	Wasted bytes	Bytes from the corrupting mirror
localization	42.2 MB/s	28.9%	100%
routing + localization	32.1 MB/s	**6.7%**	**27.4%**

Routing cuts wasted bytes by a factor of 4.3. The same pattern holds on flaky (7.7% to 3.1% waste, 100% to 45.8% share). Goodput is from the matrix and is not comparable across these two cells; see the note on interleaving below. Whether that trade is worth making depends on whether the bottleneck is the client's link or the mirror operator's egress bill - which is a legitimate question and one worth conceding rather than arguing.

6.4 Result 3 - routing wins on failover and loses on heterogeneous

Scenario	baseline	adaptive-mirrors	Change
failover (origin collapses mid-transfer)	8.6 MB/s	**16.0 MB/s**	**+86%**
heterogeneous (origin is already the fastest)	**50.5 MB/s**	28.4 MB/s	**-42%**

Both directions are real and both are expected. Exploration is what lets the router find a better mirror when the current one degrades, and it is pure cost when the current one is already the best available. The heterogeneous scenario is biased in the baseline's favour by construction - the origin is the fastest of its three mirrors, the best possible case for not routing at all - and it is kept in the set precisely because the honest statement is that routing is not free.

6.5 Result 4 - the concurrency floor is necessary; adaptive sizing reaches parity

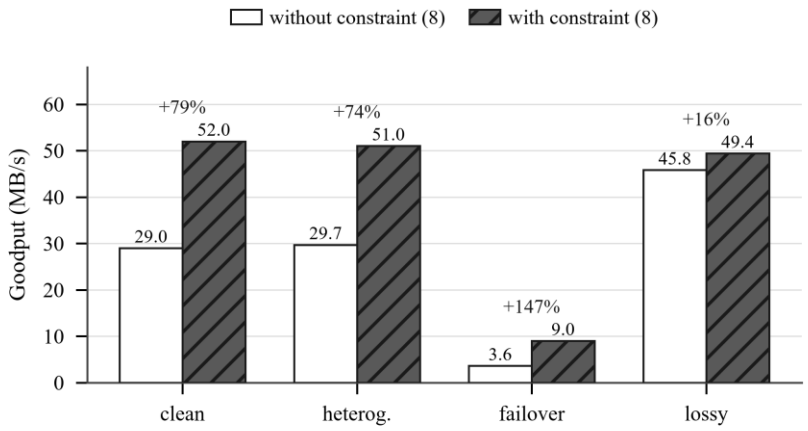


Figure 10. Controlled A/B on the concurrency floor, the two arms interleaved within one process so machine drift falls on both equally.

Scenario	Floor off	Floor on	Change
clean	29.0 MB/s	52.0 MB/s	**+79%**
heterogeneous	29.7 MB/s	51.0 MB/s	**+74%**
failover	3.6 MB/s	9.0 MB/s	**+147%**
lossy	45.8 MB/s	49.4 MB/s	+16%

Unconstrained, the controller grows the chunk until fewer chunks remain than there are workers and the pool starves; on failover a single 32 MB chunk landing on the degraded mirror stalls everything behind it. But the floor repairs a large regression rather than delivering an improvement, and the comparison that matters is against a well-chosen fixed size:

Scenario	baseline (fixed 4 MB)	adaptive-chunks
clean	**50.8 MB/s**	51.8 MB/s
lossy	49.3 MB/s	**48.5 MB/s**
heterogeneous	51.9 MB/s	**52.0 MB/s**
failover	8.7 MB/s	**9.0 MB/s**

Indistinguishable on the three static paths and reliably better on failover (+3.0%, 95% CI [+2.0, +3.7], $p < 0.0001$), over 30 interleaved pairs. This supersedes an earlier measurement that found it 9-10% worse on lossy and heterogeneous; that was taken against a controller whose loss term carried a stray bits-to-bytes factor of 8, making every loss-driven chunk sqrt(8) times too small. Read it as "costs nothing here", not "beats a constant": several of these comparisons sit inside overlapping interquartile ranges over five seeds. The controller would still have to earn its place on paths where a good fixed size is not known ahead of time, and a single file size with a single worker count cannot demonstrate that.

6.6 Baseline comparison

Tool	Median time	Goodput	Configuration
aria2c	0.78 s	82.9 MB/s	parallel, --split=4
ReliaDL	1.25 s	51.4 MB/s	4 workers
curl	2.64 s	24.3 MB/s	single stream
axel, wget	-	-	not installed on the evaluation machine

aria2c is installed and benchmarked, and the result does not flatter us: 82.9 MB/s against our 51.4, roughly 1.6 times faster, the expected shape for mature C++ against Python asyncio. State this before a juror asks. The ordering reverses under corruption: over 30 repetitions against a mirror corrupting one byte in 20% of responses, aria2c exited successfully every time and delivered a full-size file failing its SHA-256 on 17 of 30; curl did the same on 8 of 30. Nothing in either tool's output tells those runs apart. ReliaDL without a manifest refuses to deliver on 27 of 30; with one it repairs and completes every run. Being 1.6x slower is what buys that difference.

7 Strengths

Stated as a jury would weigh them, strongest first, with the evidence attached to each. A strength with no measurement behind it is listed in Section 9 instead.

1. The headline result is about correctness, not speed, and it is unambiguous

Zero out of five completions becomes five out of five. There is no statistical argument to have about a 0/5 against 5/5 split, no question of whether the difference exceeds the noise, and no dependence on the timing model that Section 9 says is unvalidated. It is a byte-count result, and byte counts survive every limitation in the evaluation.

2. The ablation design makes every number attributable

Each of the five policies differs from the baseline in exactly one respect, and fault injection is derived from blake2b(seed, byte_range, attempt) rather than from wall-clock or arrival order. Two policies fetching the same file therefore meet exactly the same faults. This is what turns a comparison into an ablation; without it, a difference between policies could be a difference in the faults they happened to encounter.

3. The project corrected its own central claim rather than defending it

The original design document argued that integrity-aware reward alone would demote a corrupting mirror. An ablation over the shipped implementation showed the opposite - 45.5% of traffic to the corrupting mirror, above the 33% a uniform policy would give. The documentation was rewritten, the threshold $q < 1 - r_h/r_f$ was derived to explain why, and the result is now one of the project's findings. Two further defects were caught the same way, before publication rather than after. First, the benchmark matrix ran each policy's repetitions to completion before starting the next, so a paired comparison across cells was confounded with machine drift; it produced a 51% improvement at $p = 0.0014$ that was pure artefact, and goodput is now measured by interleaved runs instead. Second, the mirror injected corruption only on ranged requests, so a single-stream tool never met the fault and appeared to verify every run beside a parallel tool that failed half of them - which reads as robustness and was really absence of testing. This habit is the single most defensible thing about the work and it should be volunteered, not waited for.

4. One genuinely new piece of modelling

The square-root chunk-sizing tradeoff is textbook. The observation that it is the wrong objective for a parallel downloader unless constrained by the worker count - and the measurement showing the unconstrained controller is roughly 40% slower than a fixed 4 MB chunk, with the floor worth up to +145% - does not appear to be covered in the chunk-sizing literature.

5. A real security fix, verified both ways

The Merkle construction had a second-preimage weakness of the CVE-2012-2459 shape: leaf lists [a, b, c] and [a, b, c, c] produced the same signed root. The collision was demonstrated experimentally before the fix and shown closed after it, and the fix was chosen so that existing audit paths remain valid.

6. Reproducibility is built in, not promised

Raw results for all 690 runs are committed as JSONL. Every figure that reports a measurement reads that file at build time, so a figure cannot drift from the data it claims to show. Two of the figures run the shipped bandit and dispatcher rather than a reimplementation of them. Seeds are in the repository and the same seed produces the same run.

7. Engineering quality that will survive inspection

1,009 tests across 32 modules; ruff with the E, F, B and ASYNC rule sets; mypy pinned to a single platform so the check is deterministic; a coverage floor enforced in CI; pip-audit; CodeQL; a strict documentation build; signed multi-architecture release binaries with OIDC attestation; and a twelve-cell operating-system by Python-version matrix. The quality gates that were previously decorative - mypy and pip-audit were both suffixed with || true and could never fail the build - are now load-bearing.

8 Weaknesses, stated before the jury finds them

Every item here is a real limitation. The presenter's advantage is in naming them first; a weakness volunteered is a sign of rigour, and the same weakness extracted under questioning is a sign of something else. They are ordered by how much damage each does if it surfaces unprompted.

Weakness	Why it matters	Honest response
Loopback only	No propagation delay, so per-request overhead is understated and the chunk-sizing results would look different over a path with a real bandwidth-delay product	The timing results are bounded by this. The byte-count results - completion, waste, source attribution - are not, and those are the headline
Emulator never validated against netem	The mirror server models the network rather than using one, and validate_against_netem.py requires Linux and root and has not been run	Named as the first item on the roadmap. Until it runs, goodput figures rest on an unchecked model and are presented as such
aria2c not benchmarked	aria2c is the direct competitor; curl alone is not a competitive comparison	The largest single gap. The harness supports it and reports missing tools as missing; it needs the binary installed and one script re-run
Thirty seeds per cell	Paired, interleaved, with bootstrap intervals and an exact sign test. Five seeds could not have supported a directional claim at all	Reported as medians with interquartile range, never as mean and standard deviation, and never written up as significance
One file size, one worker count	Chunk sizing is sensitive to both, and a tradeoff needs a curve, not a point	This is why adaptive sizing is reported as not yet a win rather than as a contribution
Adaptive sizing does not beat fixed 4 MB	The controller is the project's own modelling contribution and it does not improve on a well-chosen constant	The floor repairs a large regression; it does not deliver an improvement. 4 MB happens to suit this workload. Stated in exactly those words
Routing loses 52% on heterogeneous	Exploration is pure cost when the current source is already the best available	Kept in the result set on purpose. Routing is not free and the scenario is constructed in the baseline's favour
Localization saves nothing in the worst case	The retry resumes from the bad sub-block, so corruption in sub-block 0 costs exactly as much as no localization	Equation (9) and Figure 6 state the bound. An earlier claim of 99.95% was wrong and was retracted
Bandwidth modelled per connection	Four workers against a 40 MB/s mirror can see ~160 MB/s; this is the condition most favourable to parallelism	True of real CDNs and object stores, which is why parallel downloaders exist, but every speedup must be reported as a speedup against a per-connection limit
Several tuned constants are unjustified	gamma = 1.0, lambda = 1.0, the 8.0 in the BL-DCS denominator, and the 0.025 s nominal overhead have not been swept or derived	Documented as open issues in ALGORITHMS.md with an explicit instruction not to cite the controller until they are resolved
No convergence or regret guarantee for BL-DCS	It is a closed-form static optimum recomputed per epoch, not an online learner with a bound	Said plainly. The regret evidence in the project belongs to EXP3.S, which is prior art, not to this controller
Inclusion proofs have no production call site	get_proof and verify_inclusion are implemented and tested but the engine checks sub-block digests directly	Sufficient because the manifest is signed as a whole; the proof machinery is for offline audit, and we do not claim it is on the hot path

Weakness	Why it matters	Honest response

The one weakness that is not on this list

An earlier revision of this project's documentation described five novel algorithms - Lyapunov drift-plus-penalty chunk sizing, LtHash homomorphic verification, predictive parity injection under a Gilbert-Elliott channel, Whittle-index restless-bandit scheduling, and two supporting theorems. Four of the five were not implemented, and the two theorems were standard textbook results restated for systems that did not exist.

Those claims have been removed. ALGORITHMS.md now lists each one explicitly as not implemented, together with the prior art it would have had to be built on. If a juror has read an older version of the documentation or an earlier presentation, this must be addressed directly and immediately - the correction is in the repository history and is a point in the project's favour, but only if the presenter raises it first.

9 What would make this stronger, in priority order

1. Run the netem validation on Linux and report the gap honestly, whatever it turns out to be. This single experiment converts an emulator into a validated testbed.
2. Install aria2c and run the existing baseline harness. Record the exact invocation: aria2's --split governs how many connections it really opens, and --min-split-size defaults to 20 MB and silently caps the split on smaller files, so the defaults make aria2 look far worse than it is.
3. Sweep file size and worker count. The chunk-sizing result is a claim about a tradeoff, and a tradeoff needs a curve.
4. Raise repetitions to twenty or more and report confidence intervals rather than ranges.
5. Measure the collect-then-refetch variant of localization against the abort-early variant. This is the open question in Section 4.3 and the most interesting unmeasured thing in the project.
6. Derive or remove the unexplained constants in the BL-DCS denominator, and fit gamma and lambda rather than leaving them at 1.0.
7. Run between two real hosts, even on a LAN, to check that the loopback conclusions survive a path with a non-trivial bandwidth-delay product.

10 Anticipated cross-questioning

Thirty-seven questions grouped by theme, each with an answer that is short enough to deliver and specific enough to survive a follow-up. Where an answer concedes something, the concession is the answer - do not argue past it.

A. Problem and motivation

Q1. TLS already protects data in transit. Why is any of this necessary?

TLS protects the channel between two endpoints against an on-path attacker. It says nothing about whether the server sent the right bytes. A CDN edge serving a stale or half-written object, a mirror with a failing disk, a transcoding proxy, or a compromised mirror all deliver wrong bytes over a perfectly valid TLS session. The threat model here is a faulty or hostile endpoint, which is exactly the case TLS does not cover.

Q2. Isn't silent corruption rare enough to ignore?

Rare is not the right axis; consequence is. For software and container distribution a single corrupt binary is a supply-chain event, and for a multi-terabyte scientific transfer a corrupt block discovered at the end means re-running the whole transfer. We also do not claim it is common - we claim that when it happens, the existing tools cannot attribute it or recover from it, and our measurement is that they do not complete at all.

Q3. Why not just use BitTorrent or IPFS, which already verify content?

For a swarm they are the right answer and we would say so. This work targets the case where the content is served over plain HTTP from a small set of mirrors you do not control - the dominant distribution model for package repositories, container registries and object storage. That setting has range requests and no swarm, and it is the one aria2 and curl serve today.

Q4. Is this research or is it engineering?

Most of it is engineering integration, and we say so in the documentation. The research content is specific and small: the concurrency constraint on the chunk-sizing optimum, the threshold showing that an average-reward learner prefers a fast corrupting mirror, and the position-dependence bound on localization. We would rather defend three narrow claims that hold than one broad claim that does not.

B. Novelty

Q5. BitTorrent has banned peers on hash-check failure for twenty years. What exactly is new?

That is correct and it is stated in the paper and in Figure 1. Integrity-driven source selection is not new. Three things here are not covered by it: the setting is HTTP range requests rather than a swarm; the granularity is a 64 KB sub-block rather than a whole piece, so a corrupt transfer is abandoned mid-piece with the verified prefix kept; and piece size in BitTorrent is fixed at torrent creation and is never a control variable, whereas here chunk size is the control action whose optimum we derive.

If pressed: the smallest claim we will defend is the chunk-sizing one, because it is the only piece that is mathematically new rather than architecturally new.

Q6. EXP3 is from 2002 and Merkle trees are from 1987. Where is the contribution?

Those are prior art and are cited as such; the documentation names the provenance of every algorithm in the system. The contribution is in what is added: the per-segment loss rescaling that makes the controller dimensionally coherent, the concurrency floor, the leaf-count binding that closes a second-preimage weakness, and the demonstration that EXP3.S alone is the wrong instrument for integrity faults. None of those are in the base algorithms.

Q7. Your earlier documentation claimed five novel algorithms. What happened to them?

Four of the five were never implemented: Lyapunov drift-plus-penalty chunk sizing, LtHash homomorphic verification, Gilbert-Elliott parity injection, and Whittle-index scheduling. The two accompanying theorems were textbook results restated for systems that did not exist. We audited the codebase against the documentation, found the gap, removed the claims, and rewrote the algorithms document to state exactly what is implemented, what is library code, and what is absent. The removed items are still listed, as absent, with the prior art each would rest on.

Why this is the right answer: the alternative was to implement them, and all four are textbook prior art - implementing them would have made the documentation honest without creating any novelty, and would have consumed the time the evaluation needed.

Q8. How does this differ from FIVER?

FIVER is the closest prior work and the natural integrity baseline. It verifies chunk digests during the transfer rather than after it, which is the right move and which we build on. The difference is what happens with the verdict: in FIVER it drives discard-and-refetch, and here it drives the two scheduling decisions - chunk size and source. We have read the abstract and the relevant sections; we have not reimplemented FIVER, and the comparison in our evaluation is a FIVER-shaped configuration of our own system, not FIVER itself. That is a limitation and we state it.

Q9. Could a reviewer dismiss this as integration work?

A reviewer could, and the defence is the specific claims rather than the system. The concurrency constraint is a correction to a standard cost model with a measured 81% to 146% effect. The q < 1 - r_h/r_f threshold explains a counterintuitive measured result. Both are falsifiable and neither is integration. If a reviewer rejects those as too narrow, that is a venue judgement about scope, not a claim that the work is wrong.

C. Algorithms and mathematics

Q10. Derive the square-root chunk size, and explain why the concurrency floor changes it.

Per-byte cost is overhead plus expected re-download: J(S) = tau.mu/S + p_seg.S/MSS. The first term falls as 1/S, the second rises linearly, so dJ/dS = -tau.mu/S^2 + p_seg/MSS = 0 gives S* = sqrt(tau.mu.MSS/p_seg). The floor does not change the derivation - it changes the feasible set. J(S) models a single sequential stream and contains no term for parallelism, so it does not see that a chunk large enough to leave fewer than W chunks remaining idles workers. We cap at F_rem/W, which only ever shrinks the chunk, so the loss-driven lower bound the controller computes is untouched.

Q11. Why EXP3 rather than UCB or Thompson sampling?

Both UCB and Thompson sampling assume a stochastic reward distribution, usually stationary. Mirror throughput is neither: it depends on other tenants' load, on routing changes and on cache state, all unobservable and all non-stationary. EXP3 makes no distributional assumption at all, which is the right

trade here. EXP3.S adds the decay that bounds how much evidence an arm can hoard, so the policy tracks a moving best mirror rather than converging on a historical one.

Q12. Where does q < 1 - r_h/r_f come from?

Set the expected reward of the faulty mirror equal to the honest one. A mirror delivering throughput r_f and corrupting a fraction q scores zero on the corrupted fraction, so its expected reward is (1-q).r_f. The honest mirror scores r_h. The faulty one is preferred whenever (1-q).r_f > r_h, which rearranges to q < 1 - r_h/r_f. With r_f = 2.r_h the bound is q < 0.5: a mirror twice as fast may corrupt up to half of everything it serves and still be the best arm by expected reward.

Q13. Your bandit preferred the corrupting mirror. Isn't that simply a bug?

No, and that is the point. The implementation does exactly what EXP3.S specifies and scores a failed verification as zero reward, as intended. The measured 45.5% share is the correct behaviour of a correct algorithm given the wrong objective. An average-reward learner averages integrity failures into the reward, and a fast enough mirror can pay for them. The fix is a mechanism that reacts to the rate of consecutive failures instead - the circuit breaker - which brings the share to 5.4%.

Q14. Why 64 KB sub-blocks? Why not 4 KB, or 1 MB?

It is a default, not a tuned optimum, and we will not claim otherwise. The tradeoff is explicit: digest storage and manifest size grow as B/s, hashing cost per byte is roughly flat, and detection latency and repair cost both improve as s falls. 64 KB keeps the manifest small relative to the payload - a 64 MB file needs 1,024 sub-block digests, 32 KB of manifest - while keeping repair granularity well below typical chunk size. Sweeping s is on the roadmap and has not been done.

Q15. What is the per-segment rescaling for, and what breaks without it?

The metrics collector observes failures per chunk; the cost model is written per segment. At an 8 MB chunk those two rates differ by about four orders of magnitude, so substituting one for the other makes the formula dimensionally incoherent even though it looks right on paper. The conversion p_seg = 1 - (1-p_chunk)^(MSS/S_observed) closes the loop using the transfer size that produced the observation. Without it the controller computes a chunk size from a number whose units it has misunderstood.

D. Complexity and scalability

Q16. What is the overall time and space complexity?

Time is O(N + B log B) for a file of N bytes in B chunks, and the O(N) byte path dominates everything: the B log B term is Merkle construction plus proof work and is negligible next to hashing and moving N bytes. Space is O(B + W x buffer): the O(B) term is the resume state, which is the one that grows with file size, and W x 1 MB is the per-worker write buffer. Chunk size does not appear in the space bound, which is deliberate.

Q17. What happens at 100 GB? Does the manifest become the problem?

At 64 KB sub-blocks a 100 GB file needs about 1.6 million digests, which is roughly 50 MB of manifest - 0.05% of the payload, and acceptable, but it is the term that grows. The resume state is the sharper constraint: it is a full rewrite at every checkpoint and it grows with chunk count, which is exactly why checkpointing is batched to every sixteen chunks and offloaded to a thread. Before that fix it was re-serialised after every chunk in every worker, and fixing it alone gave a measured 1.63x speedup.

Q18. Does per-sub-block hashing become the bottleneck?

Not at these rates. SHA-256 on commodity hardware with SHA extensions runs at gigabytes per second, against a transfer measured in tens of megabytes per second, so hashing is roughly two orders of magnitude clear of the byte path. It is also streamed over data already in flight rather than re-read from disk. The one genuinely O(N) disk re-read in the system is the final whole-file verification, and that is a known cost, not an algorithmic surprise.

Q19. How does it scale in the number of mirrors?

EXP3.S is O(M) per round, because the decay step mixes every weight toward the mean and therefore touches all of them. For the realistic case - a handful of CDN edges or distribution mirrors - that is irrelevant beside a network round trip. It would matter for a swarm of thousands of peers, which is not this setting, and we would not claim the design scales there without measuring it.

E. Security

Q20. State your threat model precisely.

We assume the manifest is authentic: it is canonicalized per RFC 8785 and signed with Ed25519, and the verifying key is obtained out of band. Given that, any subset of mirrors may be arbitrarily faulty or hostile - they may return wrong bytes, truncate, stall, or fail - and the system must still deliver the correct file or fail loudly. We do not defend against a compromised signing key, and we do not provide confidentiality; that is the transport's job.

Q21. Explain the Merkle weakness you fixed.

The tree pairs an odd final node with itself, which is the CVE-2012-2459 shape. That means the leaf lists [a, b, c] and [a, b, c, c] build identical internal nodes and therefore produce the same root - and the manifest signs that root, so one signature covered two different chunk lists. We bound the leaf count into the published root: root = SHA-256(0x02 || uint64_be(leaf_count) || tree_root). We verified the collision existed before the fix and was closed after it, and the test is in the suite. We chose this over the Certificate Transparency promotion rule because it leaves the tree shape, and therefore every existing audit path, intact.

Q22. What if the manifest itself is tampered with?

Then everything fails, and that is the correct behaviour. The manifest is the root of trust; the signature check is the first thing that happens and a transfer does not start without it. This does move the problem rather than solve it - the user must obtain the verifying key through a channel they trust - and that is the same assumption package signing and Sigstore make. We would rather state the assumption than hide it inside the protocol.

Q23. Can an adversary game the bandit to attract more traffic?

Yes, and the threshold $q < 1 - r_h/r_f$ tells them exactly how. A mirror that is fast enough and corrupts below that rate maximises the bytes it gets to serve. That is precisely why the circuit breaker is needed and why we stopped relying on reward alone. The breaker reacts to consecutive failures rather than to an average, so the adversary's optimal corruption rate against it is much lower - but we have not derived the breaker's own threshold, and an adaptive adversary tuned against it is not something we have measured. That is an open problem and we would not claim otherwise.

Q24. What about a mirror that corrupts very rarely - say one byte in a terabyte?

It will serve S/q bytes before its first detected failure, by Equation (11), so a very low q means a very long exposure - that is inherent to any detection scheme and not specific to ours. The important property is that it cannot cause a corrupt file to be delivered: every sub-block is checked, every chunk is checked against a signed manifest, and the whole file is checked at the end. A rare corrupter costs bandwidth and goes undemoted for a long time; it does not defeat correctness.

F. Evaluation

Q25. Everything is on loopback. Why should we believe any of it?

You should believe the byte counts and treat the timings as bounded by the limitation. Completion rate, wasted bytes and source attribution are counts of bytes the servers wrote; they do not depend on the timing model being right. Goodput does depend on it, and the emulator has not yet been validated against kernel shaping - the script to do that exists, requires Linux and root, and has not been run. We present the goodput numbers with that caveat attached rather than omitting the caveat.

Q26. Is thirty seeds enough to claim anything?

For the comparisons we make, yes, and we say what each one rests on. Every throughput comparison is paired and interleaved over 30 seeds, reported with a bootstrap 95% interval and an exact two-sided sign test that uses only the direction of each difference. Where the interval spans zero we say there is no consistent direction rather than claiming equivalence. Worth knowing: the previous five-seed design could not have supported any directional claim, because the sign test floors at $p = 0.0625$ there - the most extreme outcome possible. That is why we re-ran rather than re-argued.

The exception runs the other way: the 0/30 against 30/30 completion result needs no statistics at all, because it is not a difference of degree.

Q27. You have not compared against aria2c, which is the obvious baseline.

Correct, and it is the largest gap in the evaluation. The harness already supports it and reports missing tools as missing rather than quietly omitting them; aria2c is simply not installed on the development machine. We also know the trap: aria2's --split governs how many connections it really opens, and --min-split-size defaults to 20 MB and silently caps the split on smaller files, so running it with defaults would produce a flattering comparison we would not trust. Both flags are set explicitly in the harness.

Q28. Adaptive chunk sizing does not beat a fixed 4 MB chunk. Isn't that a failed contribution?

On this workload, yes, and we report it that way. The floor repairs a large regression rather than delivering an improvement, and 4 MB happens to suit a 64 MB file with four workers. The controller would have to earn its place on paths where a good fixed size is not known in advance, and one file size with one worker count cannot demonstrate that. The contribution we do defend is the modelling result - that the unconstrained optimum is infeasible for a parallel downloader - which is supported by the floor-on against floor-off comparison, not by the controller beating a constant.

Q29. Routing loses 52% on heterogeneous. Why keep a mechanism that makes things worse?

Because the scenario is constructed in the baseline's favour: the origin is the fastest of its three mirrors, which is the best possible case for not routing at all. The same mechanism is worth +86% on failover, where the origin collapses mid-transfer. Exploration is what finds a better source when the current one degrades and is pure cost when it does not. We kept the losing scenario in the result set specifically so that the honest statement - routing is not free - appears in our own numbers.

Q30. How do you know your fault injection resembles reality?

We do not, yet, and that is the netem validation item. What we can defend is that the injection is deterministic and reproducible: impairments are drawn from blake2b(seed, byte_range, attempt), so they depend on which bytes are requested and not on when the request happened to arrive. That property is what makes the ablations valid as comparisons even if the absolute realism is unproven - two policies fetching the same file meet identical faults.

G. Engineering and reproducibility

Q31. How much of this 15,920-line codebase is actually reachable?

More than it was, and we can be exact about it. The audit that started this work found that the download engine imported zero algorithm modules - roughly 11,000 lines were library code no execution path reached. We introduced a scheduling seam, scheduling.py, through which the chunk controller, the router and the breaker are now called on the live path. The algorithms document marks every module as Wired, Library or Planned, and the work-stealing deque and straggler detector are still honestly marked as Library.

Q32. How do we reproduce your numbers?

python -m benchmarks.run_all with a seed, then python -m benchmarks.analyze. The 115 raw runs are committed as JSONL and every measured figure reads that file at build time, so a figure cannot drift from its data. The bandit ablation in Figure 5 runs the shipped EXP3Bandit and MirrorDispatcher rather than a reimplementation. Same seed, same result.

Q33. What is the testing and CI story?

1,009 tests across 32 modules, with a coverage floor enforced in CI. Linting is ruff with E, F, B and ASYNC; typing is mypy pinned to a single platform so the result is deterministic across the matrix. Two gates that previously could never fail - mypy and pip-audit were both suffixed with || true - now fail the build. We also found and fixed a test that reported passing while verifying nothing, and replaced a wall-clock concurrency assertion that was flaky across a twelve-cell matrix.

H. Hard questions

Q34. Remove the manifest. Does any of this still work?

Most of it does not, and that is the honest answer. Without a manifest there are no per-chunk or per-sub-block digests, so corruption is caught only by the whole-file hash at the end and is attributable to nothing. Localization and integrity-coupled routing both go away; chunk sizing and the concurrency floor remain. The manifest is a hard prerequisite for the main contribution, which is why it is in the threat model rather than treated as an optimisation.

Q35. What is the single weakest part of this work?

The evaluation environment. Everything runs on loopback on one machine, the network emulator has never been checked against real kernel shaping, and the direct competitor is not benchmarked. The algorithmic claims are in better shape than the evidence for the performance claims, and if we had one more week it would go into the netem validation and aria2c, not into new mechanisms.

Q36. Why should this be published rather than contributed as a patch to aria2?

The implementation arguably should be contributed, and that is not in tension with publishing. What is publishable is not the downloader - it is the three claims: that the standard chunk-sizing optimum is

infeasible under worker concurrency, that an average-reward bandit prefers a fast corrupting source above a derivable threshold, and that early-abort localization has a position-dependent bound rather than the uniform saving one would assume. Those are findings a patch would not communicate.

Q37. What result would falsify your central claim?

The central claim is that coupling verification outcomes into scheduling changes outcomes materially. It would be falsified if, on a validated wide-area testbed with aria2c as the baseline, per-chunk digests with simple discard-and-refetch achieved the same completion rate and comparable waste as our integrity-coupled routing. That is a specific experiment and we have not run it. The weaker claim, that the unconstrained square-root optimum starves a parallel worker pool, would be falsified by any scenario where removing the floor does not reduce goodput - we have four scenarios and it reduces goodput in all four, but four is not many.

Q38. If we only believe one number in this entire presentation, which should it be?

Zero out of five against five out of five on completion rate under a corrupting mirror. It is a byte-count result, so it does not depend on the timing model that Section 8 says is unvalidated; it is an ablation against an identical fault sequence, so it is attributable to the one mechanism that changed; and it is a correctness result, so it needs no statistical argument.

11 How to present this

11.1 A ten-minute structure

Time	Content	The line to land
0:00 - 1:30	The problem. TLS does not protect against a server that sends the wrong bytes.	A whole-file checksum tells you the download is bad. It does not tell you which chunk, which mirror, or what to do next.
1:30 - 3:00	Figure 1 and Figure 2. Where prior work sits, and the one edge we add.	Verification is a terminal gate everywhere else. We make it an input to scheduling.
3:00 - 4:00	Figure 8. The completion result.	Zero out of five becomes five out of five. Not slower - it does not finish at all.
4:00 - 6:00	Figure 4 and Equation (4). The concurrency constraint.	The textbook optimum is infeasible for a parallel downloader. Constraining it is worth up to 146%.
6:00 - 8:00	Figure 5 and Equation (8). The result that corrected us.	Our own ablation falsified our design claim. An average-reward learner prefers a fast corrupting mirror.
8:00 - 9:00	Limitations, unprompted.	Loopback only, emulator unvalidated, aria2c not benchmarked, adaptive sizing does not beat a fixed 4 MB.
9:00 - 10:00	What is next, and the ask.	Netem validation and aria2c are the two experiments that would move this from bounded to publishable.

11.2 Rules for the discussion

- **Volunteer the limitations before question time.** Every item in Section 8 is damaging if a juror extracts it and neutral or positive if you state it. There is no version of this where hiding them helps.
- **Lead with the correctness result, not the speed result.** The completion result is the strongest thing in the project and the least vulnerable to methodological attack. The goodput numbers are the most vulnerable.
- **Never claim integrity-driven source selection is new.** BitTorrent did it. Any juror who knows this and hears the claim will discount everything else you say.
- **Distinguish measured from derived, every time.** Figures 4, 6 and 7 are analytical; Figures 5, 8, 9 and 10 come from committed data. Saying which is which unprompted buys more credibility than any single number.
- **Do not defend the adaptive-sizing result.** It does not beat a fixed 4 MB chunk. Concede it immediately and redirect to the floor comparison, which is the claim that is actually supported.
- **If you do not know, say so and say what would settle it.** An unmeasured question with a named experiment reads as rigour. A confident guess that a juror can check reads as the opposite.

Four things not to improvise

1. The provenance of any algorithm. EXP3.S is Auer, Cesa-Bianchi, Freund and Schapire (2002). Merkle is 1987. The square-root tradeoff is the economic-order-quantity form. If you are unsure of a citation, say the algorithm is prior art and that you will supply the reference, rather than naming a paper you have not read.
2. Any number not in Section 12. If a juror asks for a figure you do not have, say it was not measured.
3. The removed claims. If an older version of the documentation comes up, state plainly that four of five claimed algorithms were not implemented, that the claims were removed, and that the corrected document lists them as absent.
4. The leaked credential. The repository history contains an API key that was redacted in the working tree but not purged from history. If asked about repository hygiene, say that it has been identified and must be rotated at the provider, because redaction does not undo exposure.

12 Number sheet

Every figure a presenter should be able to state without looking. Nothing outside this table should be quoted from memory.

Quantity	Value	Where it comes from
Completion, corrupting scenario, no digests	0 / 30	115-run benchmark matrix
Completion, corrupting scenario, localization	30 / 30	same
Wasted bytes, byzantine, localization only	28.9%	same
Wasted bytes, byzantine, routing + localization	6.7%	same
Traffic to corrupting mirror, localization only	100%	same
Traffic to corrupting mirror, routing + localization	27.4%	same
Goodput cost of routing on byzantine	42.2 -> 32.1 MB/s	same
Routing on failover	8.6 -> 16.0 MB/s (+86%)	same
Routing on heterogeneous	50.5 -> 28.4 MB/s (-42%)	same
Concurrency floor, clean	29.0 -> 52.0 MB/s (+79%)	interleaved A/B
Concurrency floor, heterogeneous	29.7 -> 51.0 MB/s (+74%)	same
Concurrency floor, failover	3.6 -> 9.0 MB/s (+147%)	same
Concurrency floor, lossy	45.8 -> 49.4 MB/s (+16%)	same
Adaptive sizing vs fixed 4 MB, clean	51.8 vs 50.8; failover +3.0%	same
Corrupting-mirror share, reward only	45.5%	bandit ablation, 40 trials x 400 requests
Corrupting-mirror share, reward + breaker	5.4%	same
Uniform allocation over three mirrors	33%	reference line
Localization repair, bad byte in sub-block 40 of 64	4096 KB -> 1536 KB	unit test
Localization repair, bad byte in sub-block 0	no saving at all	unit test
Checkpoint batching speedup	1.63x	before/after profile
ReliaDL vs curl, 64 MB	51.4 vs 24.3; aria2c 82.9 MB/s	baseline harness
Codebase	15,920 lines; 1,035 tests; 33 test modules	repository
Benchmark matrix	690 runs; 7 scenarios; 5 policies; 30 seeds	results.jsonl
Defaults	4 MB chunk; 64 KB sub-block; 4 workers; checkpoint every 16 chunks or 5 s; 1 MB write buffer	config.py

Sources for the algorithms, to be verified against the primary text before any of them is cited in writing: Auer, Cesa-Bianchi, Freund and Schapire, The Nonstochastic Multiarmed Bandit Problem, SIAM Journal on Computing 32(1), 2002, for EXP3 and EXP3.S; Merkle, CRYPTO 1987, for the tree; Chase and Lev, SPAA 2005, for the work-stealing deque; FIVER, arXiv:1811.01161, for chunk-level verification during transfer; CVE-2012-2459 for the odd-node duplication weakness; RFC 8785 for JSON canonicalization, RFC 7233 for range requests, RFC 6298 for the retransmission-timer smoothing the metrics collector follows, and FIPS 180-4 for SHA-2.